

0.85. Plusieurs ennemis : un tableau de struct

Trois B, rangés dans `Position ennemis[3]`, partis d'un endroit au hasard ; une boucle `for` les fait tous avancer vers le A.

1. Ce qu'il faut comprendre

Trois poursuivants au lieu d'un. Plutôt que `bx`, `by`, `bx2`, `by2`, `bx3`, `by3`, on les range dans **un tableau de struct** : `Position ennemis[3]`; (le type `Position` du 0.80).

Ce qui est nouveau ici : `ennemis[i].x`. D'abord la **case** du tableau (`[i]`, l'ennemi n° i), **puis** la partie (`.x`, sa colonne). `ennemis[0].x` est la colonne du premier, `ennemis[2].y` la ligne du troisième.

Une boucle `for` passe sur les trois : le même code fait avancer chacun vers le A (le 0.79). Pour quatre ennemis, on changerait seulement le 3 en 4.

Au départ, au hasard (le 0.81.2) : chaque ennemi prend une colonne `hasard() % 20` sur les lignes du haut. Chaque partie commence différemment.

Ils finissent par se rejoindre sur le A : ils visent tous la même case.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2620 octets** de cartouche.

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- LE TYPE Position (le 0.80) : une colonne et une ligne ----
```

2. UN ENREGISTREMENT — POSITION

```
struct Position { uint8_t x, y; };

// ---- UN TABLEAU DE Position : trois ennemis ----
//
//   Position ennemis[3];
//   |           |           |
//   |           |           +-- [3] : trois cases, numérotées 0, 1 et 2
//   |           +----- le nom du tableau
//   +----- chaque case est une Position (un x et un y)
//
//   ennemis[0].x ennemis[0].y ← le 1er ennemi
//   ennemis[1].x ennemis[1].y ← le 2e
//   ennemis[2].x ennemis[2].y ← le 3e
```

3. UN TABLEAU — ENNEMIS

```
Position ennemis[3];
```

4. UNE VARIABLE — X

```
uint8_t x = 10;           // le A : en bas, (10, 12)
```

5. UNE VARIABLE — Y

```
uint8_t y = 12;
```

6. UNE VARIABLE — SX

```
uint8_t sx = 0;          // la direction du prochain pas d'un ennemi
```

7. UNE VARIABLE — SY

```
uint8_t sy = 0;
```

8. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // ---- LE DÉPART : chaque ennemi au hasard, en haut de l'écran ----
    // i vaut 0, puis 1, puis 2 : la boucle s'occupe de chaque ennemi à son tour.
    for (uint8_t i = 0; i < 3; i++) {
        ennemis[i].x = hasard() % 20;          // sa colonne : de 0 à 19 (le 0.81.2)
        ennemis[i].y = hasard() % 3;          // sa ligne : de 0 à 2, tout en haut
        poser(ennemis[i].x, ennemis[i].y, ALPHABET[1]); // on l'y pose : un B
    }

    // ---- LA BOUCLE DU JEU ----
    while (true) {
        image();

        if (chaque(700)) {                    // tous les 700 ms...
            for (uint8_t i = 0; i < 3; i++) { // ...chaque ennemi fait un pas vers le A
                // ennemis[i].x se lit en deux temps :
                //   ennemis[i] : d'abord la CASE du tableau, l'ennemi n° i
                //   .x          : puis la PARTIE, sa colonne
                //
                // La direction, comme au 0.79, mais avec les parties de ennemis[i] :
                sx = 0;
                if (x > ennemis[i].x) { sx = 1; }
                if (x < ennemis[i].x) { sx = -1; }
                sy = 0;
                if (y > ennemis[i].y) { sy = 1; }
                if (y < ennemis[i].y) { sy = -1; }
                if (sx != 0 || sy != 0) {
                    // un_pas range la nouvelle place dans ennemis[i].x et ennemis[i].y :
                    // la console sait ranger dans une partie de struct (le 0.80).
                    un_pas(ennemis[i].x, ennemis[i].y, ALPHABET[1], sx, sy);
                }
            }
        }

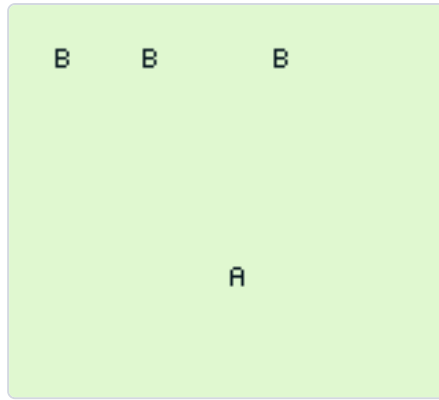
        deplace_croix(x, y, ALPHABET[0], 250); // le A, dessiné en dernier : par-dessus
    }
}
```

3. Ce qui se passe, étape par étape

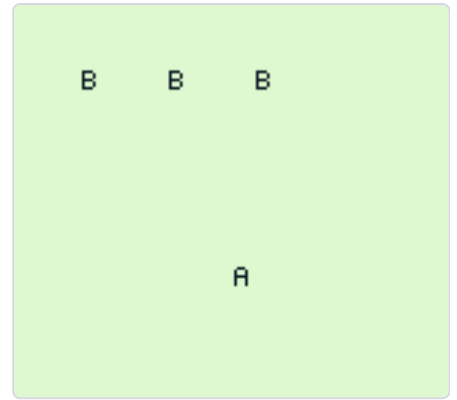
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



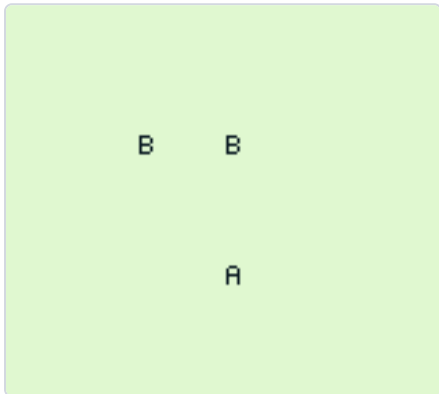
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Trois B partis d'en haut, à des places différentes à chaque partie, qui descendent tous vers le A.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ des B à l'écran au départ (3)
- ✓ ils sont descendus vers le A (ligne 8 ou plus bas)

Ensuite : **1. Écrire à l'écran** — niveau 1 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++